# CLINICAN
# Diseño documentado

Agenda de turnos, abonos, fichas y recordatorios para la peluquería canina de la veterinaria CLINICAN, en Pasto.


| | |
|---|---|
| Grupo | 5B |
| Autor | Sebastian Lopez Benavides |
| Entregables | 1 · Diseño documentado (este documento)<br>2 · Esqueleto funcional (repositorio `seba4s/Clinican`, carpeta `backend/`) |
| Versión | 2.0 · 11 de noviembre de 2026 · corresponde al backend actual |


## Contenido


1. El dominio, sus actores y sus drivers
2. Árbol de utilidad y restricciones
3. Escenarios de calidad
4. Estilo arquitectónico elegido
5. Contextos delimitados
6. Tácticas aplicadas y diseño driver por driver (ADD)
7. Diagramas C4 (niveles 1 y 2) y vista de capas
8. Decisiones de arquitectura (ADR-001 a 003)
9. Evaluación: riesgos, no-riesgos y puntos de compromiso
10. Esqueleto funcional, trazabilidad y funciones de aptitud
11. Cambios respecto a la propuesta y a la versión 1.0

## 1 El dominio, sus actores y sus drivers

CLINICAN atiende mascotas con cita previa. El personal del mostrador agenda los turnos, cobra los abonos, atiende a las mascotas y registra en una ficha lo que se hizo. El dueño de la mascota no opera el sistema: recibe un recordatorio por WhatsApp 24 horas antes del turno.

### Actores

| Actor | Tipo | Qué hace con el sistema |
|---|---|---|
| **Personal del mostrador** | Persona que opera | Recepción y peluquería son la misma gente. Agenda, mueve y cancela turnos, registra abonos e inasistencias, llena fichas, cambia precios, cupos y horario, y restaura respaldos. |
| **Dueño de la mascota** | Persona externa | No opera el sistema. Pide el turno, paga el abono y recibe el recordatorio. Su cédula es la llave del cliente y su saldo a favor pasa de un turno a otro. |
| **WhatsApp Business (Meta)** | Sistema externo | Entrega los recordatorios con una plantilla aprobada. Solo se alcanza con internet. |
| **Fichas antiguas en Excel** | Fuente externa | Historial previo de clientes y mascotas que se importa una vez. |

El negocio decidió un **solo perfil, sin roles ni PIN**: todo el personal hace todo y el programa abre directo. Lo que se sacrifica es el control de acceso (cualquiera puede cambiar un precio o restaurar un respaldo). Lo compensa en parte la auditoría: cada escritura queda registrada con qué se hizo y cuándo.

Lo que hace al dominio no trivial no son los formularios sino las reglas:

- **La capacidad de una franja depende de la mascota.** Cada franja de 30 minutos admite 2 mascotas de MÁQUINA, 1 GRANDE, 1 TIJERA y 1 COMPLICADO. La categoría sale de la mascota con la prioridad COMPLICADO > GRANDE > TIJERA > MÁQUINA.
- **Un turno no es firme hasta que se paga el abono.** Nace pendiente y tiene 30 minutos para completar $20.000. Mientras tanto ocupa cupo. Se aceptan abonos parciales; lo pagado de más queda como saldo a favor. Si vence, lo pagado vuelve a quedar a favor del dueño.
- **El dinero sigue al cliente.** El saldo a favor se aplica solo al siguiente turno; si cubre el abono, el turno nace confirmado. Cancelar deja lo pagado como saldo a favor.
- **Las inasistencias tienen consecuencias**: si el dueño avisó con 12 horas o más, el abono queda a favor; si no, se pierde.
- **El precio se fija por fecha.** La ficha toma el precio vigente en la fecha del turno; cambiar un precio crea una fila nueva con su vigencia. Todo se maneja en pesos enteros.

### Los tres drivers

| Driver | Qué exige | De dónde sale |
|---|---|---|
| **D1 · Disponibilidad sin conexión** | El personal agenda y consulta la agenda aunque no haya internet. Si la agenda no funciona, el negocio no atiende. | Condición real de operación: el local no tiene internet estable. |

| | | |
|---|---|---|
| **D2 · Integridad de la agenda** | Nunca se sobrepasa la capacidad de una franja, ni siquiera cuando dos personas piden el último cupo a la vez o un pendiente vence mientras se registra su abono. | Es la regla que más afecta el servicio: una franja sobrevendida es un cliente esperando o una peluquera con dos mascotas a la vez. |
| **D3 · Modificabilidad de las notificaciones** | Agregar o cambiar el canal de recordatorios (hoy WhatsApp, mañana SMS o correo) no obliga a tocar la lógica de la agenda. | La API de Meta cambia sus reglas, exige plantillas aprobadas, un token y conexión para enviar. |

### Funciones primarias y preocupaciones del arquitecto

De los casos de uso del sistema, cuatro determinan la arquitectura porque atraviesan varios módulos o estresan las reglas; el resto (buscar clientes, unir duplicados, importar Excel, ver el historial) cabe sin decisiones nuevas.

| Función primaria | Por qué estresa el diseño | Driver |
|---|---|---|
| Crear o mover un turno | Cuenta el cupo, aplica el saldo a favor y programa el recordatorio en una sola transacción. | D2, D3 |
| Registrar un abono | Compite con el vencimiento a los 30 minutos y puede dejar saldo a favor. | D2 |
| Enviar el recordatorio | Es lo único que usa la red; no puede frenar a la agenda. | D1, D3 |
| Marcar atendido y llenar la ficha | Toma el precio vigente en la fecha del turno y descuenta el abono al entregar. | — |

Preocupaciones que nadie pidió pero hay que resolver: migrar la base al instalar una versión nueva ( `datos/migraciones.py` ), respaldar y restaurar sin un técnico, probar reglas que dependen de la hora (reloj inyectado en todos los casos de uso) e instalar en Windows con un solo comando.

## 2 Árbol de utilidad y restricciones

Cada hoja es un escenario concreto marcado con **(importancia para el negocio, dificultad técnica)** en Alta, Media o Baja. Las hojas resaltadas son las que priorizamos: alta importancia y dificultad alta o media. Son cuatro de trece y solo una es (A, A); el resto se atiende, pero no dirige la estructura.

**Utilidad de CLINICAN**

- **Disponibilidad**
  - Operación sin red
    - E1 · Con el internet caído, el personal agenda un turno y ve la agenda del día en menos de 2 s. (A) (M)
  - Recuperación
    - E4 · Si la base se daña, se restaura el último respaldo en menos de 10 minutos, perdiendo como máximo el día en curso. (A) (M)
    - Si falla el envío de un recordatorio, la agenda sigue funcionando y el envío se reintenta a los 1, 5 y 15 minutos. (M) (B)
- **Integridad**
  - Capacidad de la franja
    - E2 · Dos personas piden a la vez el último cupo de una franja: solo una lo obtiene y 0 franjas quedan con más mascotas que su cupo. (A) (A)
  - Dinero
    - Abonos parciales, excedentes, cancelaciones y vencimientos dejan el saldo a favor exacto, en pesos enteros. (A) (B)
  - Ciclo de vida
    - Un turno solo cambia de estado por una transición válida (no se entrega lo que no se atendió). (M) (M)
    - Una actualización del programa migra la base sin perder registros. (M) (M)
- **Modificabilidad**
  - Canales de notificación
    - E3 · Cambiar WhatsApp por SMS se hace agregando un adaptador, sin modificar agenda ni dominio, en menos de 1 día-persona. (A) (M)
  - Reglas configurables
    - El personal cambia precios, cupos, abono y horario desde Ajustes, sin programar. (M) (B)
- **Seguridad y privacidad**
  - Trazabilidad
    - El 100 % de las escrituras queda en la auditoría con qué se hizo y cuándo. (M) (B)
  - Datos personales (Ley 1581)
    - Cédulas y celulares no salen del equipo, salvo el celular que se envía a Meta con el recordatorio. (M) (B)
- **Usabilidad**
  - Una persona nueva agenda un turno en menos de 2 minutos sin capacitación. (M) (M)
- **Rendimiento**
  - La agenda del día abre en menos de 1 s con cinco años de historial (más de 40.000 turnos). (B) (B)

A alta M media B baja Primer chip: importancia · segundo chip: dificultad hoja priorizada

## Restricciones (no se negocian)

| Restricción | Tipo | Efecto en el diseño |
|---|---|---|
| Internet inestable en el local | Condición de operación | Nada que la agenda necesite puede depender de la red. |
| Un solo computador con Windows 11 | Condición de operación | Sin servidor propio; la aplicación y los datos viven en el mismo equipo. |
| API oficial de WhatsApp (Meta) | Integración | Solo plantillas aprobadas, con 4 parámetros; requiere conexión y un token que va en variables de entorno, nunca en el código. |
| Ley 1581 de 2012 (datos personales) | Norma | Se guardan cédula y celular. Los datos se quedan en el equipo y en sus respaldos; la autorización del titular se gestiona fuera del sistema (riesgo R5). |
| Presupuesto mínimo; personal con poca experiencia en computadores | Negocio | Sin licencias ni hosting pagado; un solo perfil, letra grande y flujos cortos. |

## 3 Escenarios de calidad

Los tres primeros salen de las hojas priorizadas y son los que dirigen la estructura. E4 se incluye porque restringe decisiones que tomamos por los primeros. Cada uno tiene sus seis partes y una medida que se puede probar.

| Parte | **E1 · Agendar sin internet** (D1) | **E2 · Último cupo en disputa** (D2) | **E3 · Cambiar de canal** (D3) |
|---|---|---|---|
| **Fuente** | Personal del mostrador | Dos personas del mostrador, cada una con un cliente | El negocio, con un desarrollador |
| **Estímulo** | Agenda un turno nuevo y consulta la agenda del día | Ambas crean a la vez un turno en la única plaza de MÁQUINA que queda a las 9:00 | Decide enviar los recordatorios por SMS en vez de WhatsApp |
| **Artefacto** | Servicio de agenda y base de datos | Reglas de cupo ( `dominio/cupos.py` ) y servicio de agenda | Notificaciones: puerto, despachador y adaptadores |
| **Entorno** | Operación normal, sin conexión a internet desde hace horas | Operación normal, franja con 1 de 2 cupos de máquina ocupados | Tiempo de desarrollo, sistema en producción |
| **Respuesta** | El turno queda guardado y se muestra; ningún paso espera a la red | Una obtiene el turno; la otra recibe «No hay cupo de máquina a las 09:00» y nada queda a medias | Se escribe una clase con `enviar(mensaje)` , se registra con `registrar("sms", fabrica)` y se elige en la configuración |
| **Medida** | 100 % de los casos de uso de la agenda funcionan con la red bloqueada; agenda del día < 1 s | 0 franjas con ocupación mayor que su cupo, verificado por pruebas que reproducen la carrera con varios hilos | 0 líneas cambiadas en `dominio/` y `servicios/agenda.py` ; < 1 día-persona |

| Parte | **E4 · Base dañada** (disponibilidad) |
|---|---|
| **Fuente** | Falla del disco o un cierre brusco del equipo |
| **Estímulo** | La base de datos queda ilegible |
| **Artefacto** | Base de datos y carpeta de respaldos |
| **Entorno** | Fin de la jornada, con respaldo automático del día y del último cierre |
| **Respuesta** | El personal elige una copia en Ajustes y la restaura; antes, el sistema verifica la copia y respalda la base actual; si la copia es de una versión anterior, la migra |
| **Medida** | Restauración < 10 min; pérdida ≤ 1 día de registros; 0 restauraciones desde una copia que no pasa `PRAGMA quick_check` |

## 4 Estilo arquitectónico elegido

**Monolito de escritorio en capas, con el dominio al centro y puertos y adaptadores en el borde de integración.** Un solo proceso de Python con cinco capas ( `ui` , `servicios` , `datos` , `adaptadores` y `dominio` ) y una base SQLite embebida en el mismo equipo. Las

dependencias apuntan en una sola dirección, hacia el dominio. WhatsApp entra por un puerto definido en `servicios` y se conecta con un adaptador, con una bandeja de salida que guarda los mensajes hasta que haya red. En términos del curso, es un monolito modular: un despliegue, una base y fronteras internas reales.

| Driver | Por qué este estilo lo atiende |
|---|---|
| D1 · Disponibilidad sin conexión | Todo lo que la agenda necesita está en el mismo computador: proceso, reglas y datos. No hay salto de red en ningún caso de uso de la agenda. |
| D2 · Integridad de la agenda | Un solo proceso y una sola base con transacciones ACID permiten volver a contar el cupo dentro de una transacción con bloqueo de escritura. No hay réplicas que coordinar. |
| D3 · Modificabilidad de las notificaciones | El puerto `Notificador` aísla el canal: la agenda deja «recordatorios pendientes» y no sabe quién los envía ni cómo. |

### Qué sacrifica

- **Escalabilidad y acceso remoto.** Funciona en un solo computador. Nadie ve la agenda desde el celular ni desde otra sede, y un segundo computador no puede compartir la base de forma segura (riesgo R3).
- **Inmediatez de los recordatorios.** Salen cuando hay conexión, no en el instante en que se generan.
- **Despliegue independiente.** Cualquier cambio, aunque sea del canal, se instala como una versión completa.
- **Un punto único de falla físico:** el disco del computador. Se mitiga con respaldos (T4), no con redundancia.
- **Efecto sumidero en algunas consultas.** Agregar un campo al turno toca entidad, repositorio, servicio y pantalla, aunque dos de ellos solo pasen el dato. Lo aceptamos a cambio de poder probar cada capa por separado.

### Estilos que evaluamos y descartamos

| Alternativa | Por qué no |
|---|---|
| Microservicios (agenda, fichas, notificaciones) | Agrega la red entre servicios justo donde D1 exige no depender de ella y obliga a coordinar el cupo entre procesos (D2). Su beneficio real es que varios equipos despliegen por separado; aquí hay un equipo y un computador. |
| Aplicación web en la nube | Si se cae internet, la agenda no abre: viola D1 de entrada. |
| Web local con sincronización a la nube (*offline-first*) | Resuelve el acceso remoto, pero la sincronización con conflictos es la peor enemiga de D2: dos copias podrían dar el mismo cupo. Es la opción para crecer (R3), no para hoy. |
| Eventos con un broker de mensajes | Desacopla las notificaciones (D3), pero exige operar un broker en el PC del mostrador. La bandeja de salida en SQLite logra el mismo desacople sin otra pieza que instalar. |

## 5 Contextos delimitados

Dentro de cada capa, el código se organiza por contexto: cada uno tiene sus módulos en `dominio`, `datos` y `servicios`. Trazamos una frontera donde aparece una de las cuatro señales de la sesión 6: el vocabulario cambia, el dueño de los cambios es otro, el ritmo de cambio es distinto o la carga es distinta.

| Contexto | Responsabilidad y módulos | Señal que justifica la frontera |
|---|---|---|
| **Agenda** | Franjas, cupos por categoría, turnos, estados, abonos, vencimiento, saldo aplicado, bloqueos. `dominio/agenda.py`, `cupos.py`, `estados.py`, `abonos.py`, `servicios/agenda.py`, `datos/repo_agenda.py` | **Ritmo:** estados que cambian en minutos (pendiente, confirmado, liberado). **Vocabulario:** aquí «servicio» es solo un tipo que define la categoría de cupo. |
| **Atención** | Ficha de servicio, precio final, precios con vigencia, sesiones de desenredado. `dominio/precios.py`, `servicios/fichas.py`, `servicios/precios.py` | **Vocabulario:** «servicio» es lo que realmente se hizo, con precio y observaciones. **Ritmo:** cambia cuando cambian los precios, no la agenda. |
| **Clientes** | Propietarios (llave: cédula), mascotas, razas, saldo a favor, unión de duplicados, importación de Excel. `servicios/clientes.py`, `servicios/importacion.py` | **Ritmo:** datos de referencia que viven años; se corrigen, unen e importan sin tocar turnos. |
| **Notificaciones** | Recordatorios, bandeja de salida, despachador, canales. `dominio/notificaciones.py`, `servicios/notificaciones.py`, `servicios/despachador.py`, `adaptadores/` | **Dueño:** las reglas las cambia un tercero (Meta). **Ritmo:** otro distinto al de la agenda. Es la frontera que pide D3. |
| **Operación** | Configuración, auditoría, respaldos, migraciones. `servicios/configuracion.py`, `auditoria.py`, `respaldos.py` | **Transversal:** ninguna regla de la agenda depende de cómo se respalda o se audita. |

### Cómo se relacionan

- **Agenda → Notificaciones:** la agenda no llama al canal. Escribe un hecho en la bandeja («recordar el turno 812 el lunes a las 9:00») en la misma transacción que confirma o mueve el turno. El despachador lo lee después.
- **Agenda → Clientes:** lee la mascota y el propietario por identificador y ajusta su saldo a favor.
- **Atención → Agenda:** la ficha nace cuando el turno pasa a atendido.
- **Todos → Operación:** leen la configuración y escriben en la auditoría.
- Las pantallas leen vistas combinadas (turno con mascota y propietario) desde `datos/repo_consultas.py`, que hace joins de solo lectura sobre tablas de varios contextos.

Es un atajo consciente: en un monolito con una sola base evita N+1 consultas; si un contexto se extrajera, esa consulta sería lo primero que habría que rehacer.

## 6 Tácticas aplicadas

| Táctica | Contra | Cómo se aplica en CLINICAN | Qué degrada |
|---|---|---|---|
| **T1 · Eliminar la dependencia remota** (disponibilidad) | E1 | Base embebida (SQLite en modo WAL) y reglas locales. Solo `adaptadores/` puede importar módulos de red; lo verifica una prueba. | Acceso remoto y uso desde varios equipos. |
| **T2 · Transacción con bloqueo de escritura y reconteo** (integridad) | E2 | Toda escritura de un caso de uso va dentro de `BEGIN IMMEDIATE` (`datos/conexion.py`). El cupo se vuelve a contar dentro de esa transacción; si ya no hay, se rechaza y nada queda a medias. Espera hasta 5 s si otra conexión escribe. | Concurrencia: un solo escritor a la vez. Irrelevante con un computador; cuello de botella con muchos. |
| **T3 · Reglas en un núcleo puro y máquina de estados** (integridad, testeabilidad) | E2 | Categoría de cupo, cupos libres, abonos, precios y transiciones son funciones de `dominio/` sin base ni red, probadas aisladamente. | Algo de duplicación: la consulta trae datos y el dominio decide. |
| **T4 · Respaldo verificado y recuperación al iniciar** (disponibilidad) | E4 | Respaldo diario y al cerrar con la API de copia de SQLite, verificado con `quick_check`; se conservan los últimos 30. Al restaurar, primero se respalda lo actual. Al abrir, se migra la base y se liberan los pendientes vencidos. | Espacio en disco; el cierre tarda un poco más. |
| **T5 · Encapsular con un puerto** (modificabilidad) | E3 | El puerto `Notificador` está en `servicios/notificaciones.py`; los canales (WhatsApp, archivo, consola) en `adaptadores/`, registrados por nombre. | Una indirección más: hay que leer dos archivos para seguir un envío. |
| **T6 · Intermediario persistente (bandeja de salida)** (modificabilidad, disponibilidad) | E1, E3 | El recordatorio se guarda en `notificaciones_salida` en la misma transacción del turno. El despachador lo envía cuando llega su hora. | Latencia de entrega; entrega «al menos una vez». |
| **T7 · Reintento con espera creciente y tiempo de espera** (disponibilidad) | E1 | Si el envío falla, se reintenta a los 1, 5 y 15 minutos y luego queda FALLIDO. La llamada a Meta tiene 15 s de tiempo de espera, así que un servidor lento no cuelga al despachador. | Un recordatorio puede llegar hasta 21 minutos tarde o no llegar. |
| **T8 · Restringir dependencias** (modificabilidad) | E3 | `tests/test_arquitectura.py` falla si una capa importa a otra que no debe, si hay SQL fuera de `datos/` o red fuera de `adaptadores/`. | Más fricción al programar: no se puede tomar un atajo aunque sea más rápido. |
| **T9 · Controlar el reloj** (testeabilidad) | E2 | Ningún caso de uso lee la hora: la recibe como `ahora`. `main.py` es la única fuente de la hora real. | Un parámetro más en cada función. |
| **T10 · Auditar** (seguridad) | — | Cada escritura deja en la tabla `auditoria` qué se hizo, sobre qué y cuándo. | Más escrituras por caso de uso y una tabla que crece. |

### Diseño driver por driver (ADD)

Seguimos el método de la sesión 7: un driver a la vez, empezando por el más importante, y siempre nombrando lo que se degradó y verificando que cabe.

| Paso | Iteración 1 · D2 (A, A) | Iteración 2 · D1 | Iteración 3 · D3 |
|---|---|---|---|
| **1 · Driver** | Nunca sobrevender una franja (E2) | La agenda funciona sin red (E1) | Cambiar de canal sin tocar la agenda (E3) |
| **2 · Qué refinar** | Servicio de agenda y su almacenamiento | Todo el sistema: dónde viven proceso y datos | El envío de recordatorios |
| **3 · Tácticas** | T2 transacción con reconteo, T3 núcleo puro, T9 reloj controlado | T1 sin dependencia remota, T4 respaldos | T5 puerto, T6 bandeja de salida, T7 reintentos, T8 restringir dependencias |
| **4 · Responsabilidades e interfaces** | `categoria_de_cupo()` y `transicionar()` en dominio; `crear_turno()` y `registrar_abono()` abren la transacción | SQLite en el mismo equipo; solo `adaptadores/` toca la red | `Notificador.enviar(mensaje)`; `registrar(nombre, fabrica)`; `main.py` elige el canal |
| **5 · Verificar y qué se degradó** | Cumple (`test_carreras.py`). Degrada la concurrencia: un escritor a la vez; con un PC cabe | Cumple (`test_sin_red.py`). Degrada el acceso remoto; el negocio no lo pide hoy | Cumple (parte 4 de la demo). Degrada la inmediatez y permite duplicados raros; un recordatorio 24 h antes lo tolera |
| **6 · Siguiente** | D1, porque D2 ya fijó una sola base | D3, porque el único uso de la red es el recordatorio | Todos los drivers (A, ·) atendidos: se detiene |

La primera iteración limitó a las siguientes: al fijar una sola base con un escritor, D1 se resolvió con una base local y no con réplicas, y D3 tuvo que guardar sus mensajes en esa misma base.

# 7 Diagramas C4

## Nivel 1 · Contexto del sistema

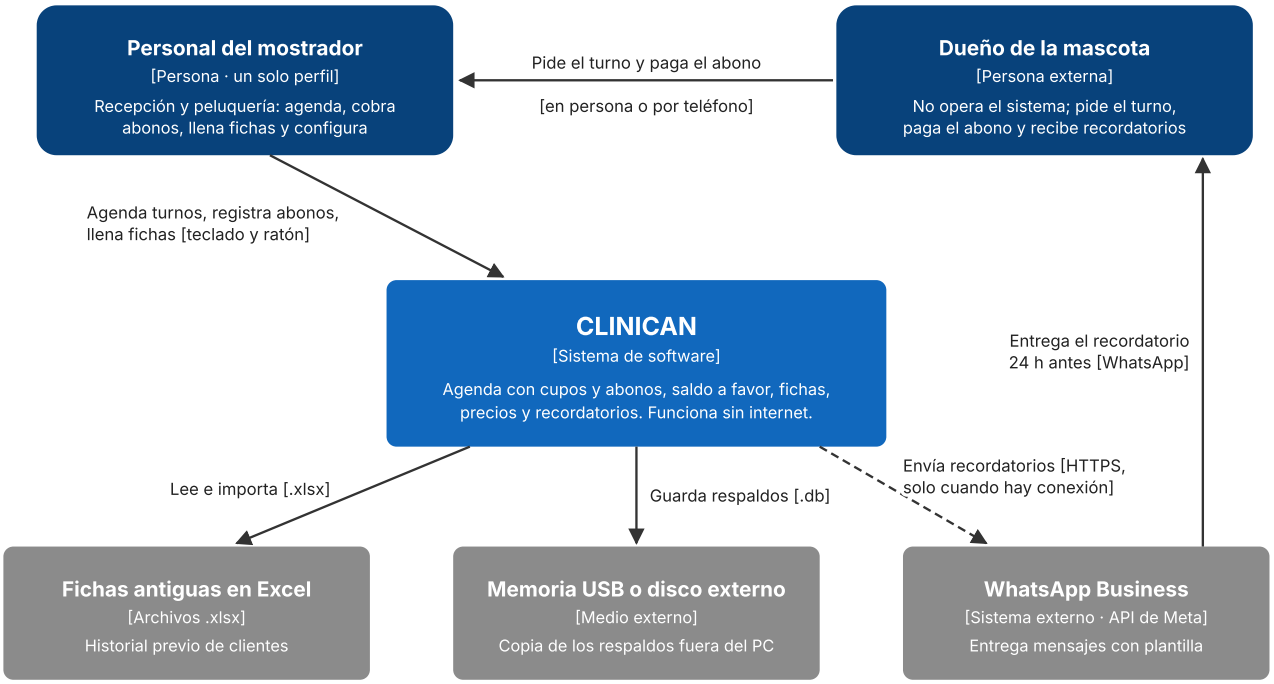



**Figura 1.** Contexto. Leyenda: azul oscuro = persona; azul = el sistema que diseñamos; gris = sistema o medio externo. Flecha continua = interacción local o en persona; flecha discontinua = depende de internet y es asíncrona (no bloquea la agenda).

## Nivel 2 · Contenedores

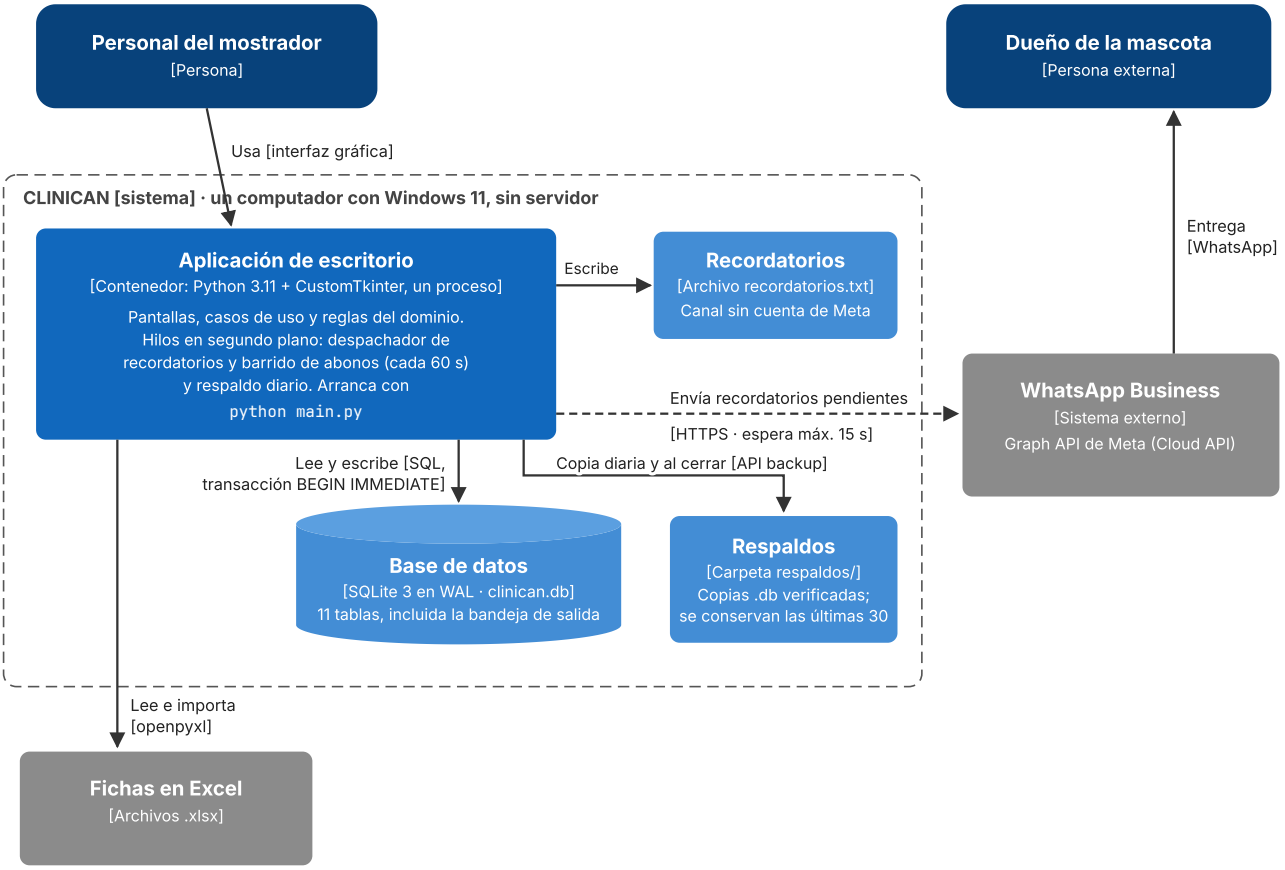



**Figura 2.** Contenedores. Leyenda: azul fuerte = contenedor que ejecuta código; azul claro = almacenamiento; gris = externo; borde discontinuo = frontera del sistema (un solo computador). Flecha discontinua = sale por internet de forma asíncrona. Ninguna flecha de la agenda cruza la red. La base vive en `%USERPROFILE%\Clinican\clinican.db` (o en la ruta de `CLINICAN_BD`).

## Vista complementaria · capas dentro de la aplicación

Es la vista de desarrollo (4+1) que el esqueleto demuestra: cómo se reparten las responsabilidades dentro del contenedor «Aplicación de escritorio» y hacia dónde apuntan las dependencias.

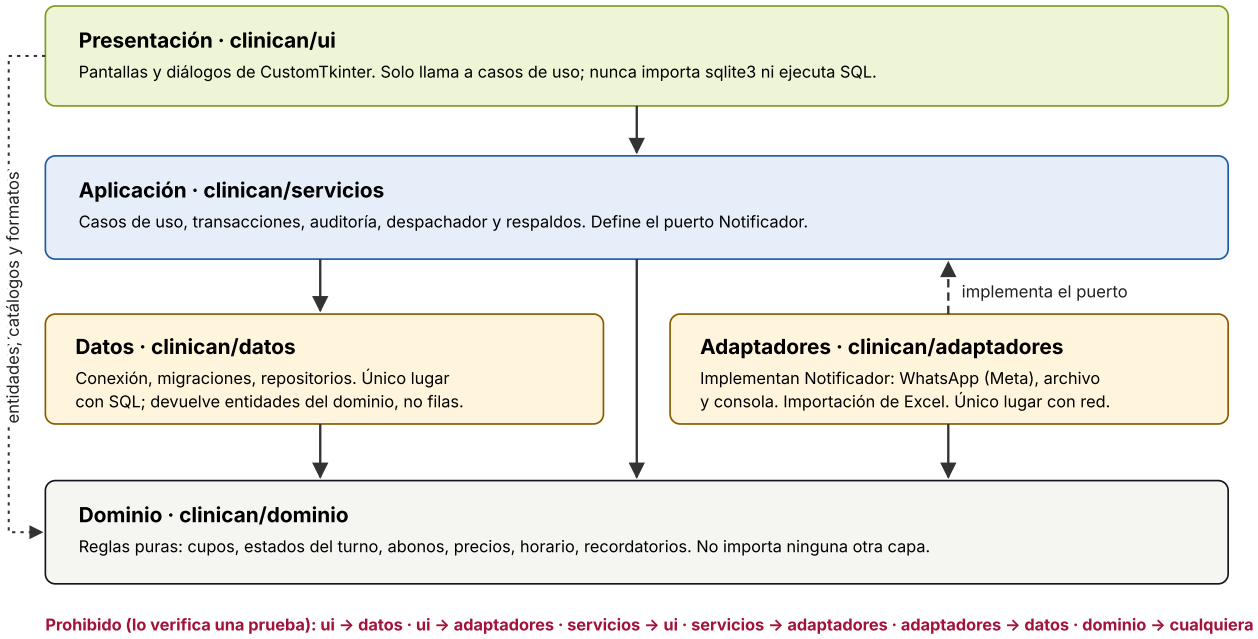



**Figura 3.** Capas. Flecha = «depende de» (importa). Todas apuntan hacia el dominio. La flecha punteada de la izquierda es la única que salta una capa: la presentación usa del dominio entidades, catálogos y formatos para mostrar, nunca reglas con efectos. Los adaptadores se conectan en el arranque ( `main.py` , la raíz de composición).

## Acoplamiento medido por capa

Contando las capas que dependen de cada una (Ca) y de las que depende (Ce), con I = Ce / (Ca + Ce). La raíz de composición ( `main.py` , `demo.py` ) no cuenta. Las dependencias apuntan hacia lo estable, como pide la sesión 4.

| Capa | Ca (quién depende de ella) | Ce (de quién depende) | I | Lectura |
|---|---|---|---|---|
| `dominio` | 4 · datos, servicios, adaptadores, ui | 0 | 0 | La más estable; cambiarla es caro y por eso no depende de nada |
| `datos` | 1 · servicios | 1 · dominio | 0,5 | Intermedia |
| `servicios` | 2 · adaptadores, ui | 2 · dominio, datos | 0,5 | Intermedia; define el puerto que implementan los adaptadores |
| `adaptadores` | 0 | 2 · dominio, servicios | 1 | Inestable a propósito: aquí cambia el canal |
| `ui` | 0 | 2 · dominio, servicios | 1 | Inestable a propósito: las pantallas cambian sin romper a nadie |

## 8 Decisiones de arquitectura

Cada ADR tiene título, estado con fecha, contexto, decisión, alternativas consideradas y consecuencias buenas y malas. Los mismos textos están versionados junto al código en `docs/adr/`.

### ADR-001 · Monolito de escritorio en capas con base de datos local

Estado: Aceptada · 2026-11-11

#### CONTEXTO

D1 exige que la agenda funcione sin internet: el local no tiene conexión estable y, si la agenda no abre, no se atiende. La restricción de operación es un solo computador con Windows 11, sin presupuesto para servidor ni hosting. Lo usa el personal del mostrador con un solo perfil. D2 pide que nunca se sobrevenda una franja, lo que es más fácil de garantizar con una sola base.

#### DECISIÓN

Construiremos una aplicación de escritorio en Python, un solo proceso, con cinco capas (`ui`, `servicios`, `datos`, `adaptadores`, `dominio`) y SQLite embebido. Las dependencias van en una sola dirección, hacia el dominio, y las verifica `tests/test_arquitectura.py`.

#### ALTERNATIVAS CONSIDERADAS

- **Aplicación web en la nube.** Sin internet no hay agenda: viola D1.
- **Microservicios** (agenda, fichas, notificaciones). Mete la red donde D1 pide no depender de ella y obliga a coordinar el cupo entre procesos (D2). Con un equipo y un PC se pagan los costos sin el beneficio.
- **Servidor local con cliente web** (Flask en el mismo PC). Cumple D1, pero agrega un servidor que levantar y vigilar sin ganancia hoy.

#### CONSECUENCIAS

- Buena: ningún caso de uso de la agenda cruza la red (`tests/test_sin_red.py`).
- Buena: las transacciones ACID locales hacen posible ADR-002 sin coordinación distribuida.
- Mala: no hay acceso remoto ni uso desde varios computadores; compartir el archivo por red puede corromperlo (R3).
- Mala: el disco del PC es un punto único de falla; dependemos de los respaldos (R1).
- Mala: todo cambio se instala como una versión completa.

### ADR-002 · Cupos por categoría contados dentro de una transacción con bloqueo

Estado: Aceptada · 2026-11-11 · reemplaza el modelo de la propuesta (agenda por peluquera)

### CONTEXTO

D2: nunca sobrevender. El negocio no asigna peluquera al agendar; trabaja con franjas fijas de 30 minutos y lo que varía es cuántas mascotas caben por categoría: MÁQUINA 2, GRANDE 1, TIJERA 1, COMPLICADO 1. Los pendientes de abono no vencidos ocupan cupo. Hay carreras reales: dos personas piden el último cupo a la vez, y un pendiente puede vencer justo cuando se registra su abono.

### DECISIÓN

- La categoría, los cupos libres y las transiciones de estado son funciones puras del dominio.
- Cada caso de uso que ocupa un cupo (crear, mover) abre `BEGIN IMMEDIATE`, vuelve a contar la ocupación dentro de la transacción y escribe solo si cabe.
- Registrar un abono revisa el vencimiento en la misma transacción: si venció, libera el turno y devuelve lo pagado como saldo a favor.

### ALTERNATIVAS CONSIDERADAS

- **Agenda por peluquera con duración variable** (la propuesta inicial). No refleja cómo trabaja el negocio; se descartó tras hablar con la administradora.
- **Validar el cupo solo en la pantalla.** Entre ver el cupo libre y guardar puede llegar otra persona; la validación tiene que estar donde se escribe.
- **Restricción o trigger en SQL.** No expresa «máximo N por categoría según la configuración» y escondería la regla lejos de las pruebas del dominio.
- **Bloqueo optimista con versión.** Sirve con muchos escritores; aquí hay uno y obligaría a reintentar desde la interfaz.

### CONSECUENCIAS

- Buena: la regla vive en un solo lugar y `tests/test_carreras.py` reproduce la carrera con varios hilos.
- Buena: cupos, abono y horario se cambian en Ajustes, sin programar.
- Mala: un solo escritor a la vez; con varios equipos sería el primer cuello de botella.
- Mala: no se sabe qué peluquera atiende cada turno.
- Mala: un pendiente que nunca paga bloquea el cupo hasta 30 minutos.

## ADR-003 · Recordatorios por un puerto y una bandeja de salida en la base local

Estado: Aceptada · 2026-11-11

### CONTEXTO

D3 exige cambiar el canal de recordatorios sin tocar la agenda, y D1 exige que la agenda no espere a la red. La API de Meta pide plantillas aprobadas, conexión y un token que no puede ir en el código. El recordatorio sale 24 horas antes del turno.

### DECISIÓN

- Al confirmar o mover un turno, el servicio de agenda guarda el recordatorio en `notificaciones_salida`, en la misma transacción.

- Un despachador en un hilo con su propia conexión revisa la bandeja cada 60 s. Justo antes de enviar verifica que el turno siga confirmado y no haya empezado; si no, lo descarta con motivo. Si falla, reintenta a los 1, 5 y 15 minutos; luego queda FALLIDO. Purga lo cerrado de más de 90 días.
- El despachador solo conoce el puerto `Notificador`. Los canales (WhatsApp, archivo, consola) viven en `adaptadores/` y se registran por nombre; `main.py` elige uno según la configuración.

### ALTERNATIVAS CONSIDERADAS

- **Llamar a WhatsApp desde el servicio de turnos.** Acopla la agenda al canal (viola D3) y la bloquea sin red (viola D1).
- **Microservicio de notificaciones en la nube.** Exige sincronizar la agenda hacia la nube y pagar un servidor.
- **Broker de mensajes** (RabbitMQ o similar). Mismo desacople con una pieza más que instalar en el PC del mostrador.

### CONSECUENCIAS

- Buena: un canal nuevo es una clase y una línea de registro; la agenda no cambia.
- Buena: sin red, los recordatorios esperan en la base y en sus respaldos.
- Mala: pueden llegar tarde; tras tres reintentos hay que reintentarlos a mano en la pantalla Recordatorios.
- Mala: entrega «al menos una vez»: una caída entre enviar y marcar puede duplicar el mensaje.
- Mala: el envío real depende de una cuenta de Meta verificada y una plantilla aprobada (R2).

## 9 Evaluación: riesgos, no-riesgos y puntos de compromiso

Aplicamos las cuatro salidas de ATAM (sesión 9). El resultado es una lista de cosas por verificar, no una nota.

### Riesgos

| # | Riesgo | Impacto | Qué habría que verificar para descartarlo |
|---|---|---|---|
| R1 | Los respaldos quedan por defecto en el mismo disco que la base. Un robo, un ransomware o un disco dañado se lleva todo. | Pérdida total del historial y del saldo a favor de los clientes. Afecta D1 y E4. | Simulacro: apuntar `carpeta_respaldos` a una USB, borrar la base en un equipo de prueba, restaurar y medir tiempo y pérdida. Acordar quién saca la copia del equipo y cada cuánto. |
| R2 | Meta no aprueba la plantilla, cambia la política del número o el costo por conversación hace inviable el canal. | No salen recordatorios. La agenda sigue (ADR-003), pero D3 pierde su valor hasta tener otro canal. | Crear la cuenta de Meta Business, someter la plantilla `recordatorio_turno` con sus 4 parámetros, enviar un mensaje real y consultar la tarifa vigente para Colombia. |
| R3 | El negocio pone un segundo computador abriendo el mismo archivo SQLite por la red local. | Bloqueos y posible corrupción. Es lo primero que se rompe si el negocio crece diez veces. | Probar dos equipos escribiendo a la vez sobre una carpeta compartida de Windows y medir errores. Mientras tanto, la instrucción es un solo equipo. |
| R4 | La red falta por semanas o el token vence, y la bandeja se llena de recordatorios inútiles. | Mensajes tardíos y la base creciendo. | Ya mitigado en parte: los vencidos se descartan antes de enviar y lo cerrado se purga a los 90 días. Falta un simulacro con la red cortada 30 días. |
| R5 | La Ley 1581 exige autorización del titular para tratar cédula y celular, y el sistema no la registra. | Incumplimiento legal y un recordatorio enviado sin autorización. | Confirmar con la administradora cómo se recoge hoy la autorización (por ejemplo, un formato firmado al llegar). Si no existe, agregar al cliente la fecha de autorización y que el despachador la revise. |

### No-riesgos (verificados, no se rediscuten)

- **N1 · Volumen.** Con cinco años de datos sintéticos (más de 40.000 turnos), la agenda del día y las consultas principales responden en menos de 1 s (`tests/test_rendimiento.py`).
- **N2 · Carrera por el último cupo.** Varios hilos con conexiones distintas nunca dejan una franja sobre su cupo (`tests/test_carreras.py`).
- **N3 · Agenda sin red.** Todos los casos de uso de agenda funcionan con los sockets bloqueados (`tests/test_sin_red.py`).

### Puntos sensibles

- **S1 · Tiempo de espera del bloqueo (5 s).** De él depende la usabilidad cuando dos escrituras chocan: más bajo da errores falsos de «base ocupada», más alto congela la pantalla.
- **S2 · Esperas de reintento (1, 5 y 15 min).** De ellas depende cuán tarde llega un recordatorio cuando la red vuelve.

- **S3 · Vencimiento del abono (30 min).** De él depende cuánto tiempo un pendiente bloquea un cupo que otro cliente pudo pagar.

### Puntos de compromiso

| Decisión | Mejora | Empeora | Por qué es aceptable |
|---|---|---|---|
| `BEGIN IMMEDIATE` con reconteo (ADR-002) | Integridad (D2) | Concurrencia y escalabilidad | Hay un solo PC; las escrituras duran milisegundos. |
| Bandeja de salida (ADR-003) | Disponibilidad (D1) y modificabilidad (D3) | Inmediatez; posibles duplicados | Un recordatorio 24 h antes tolera minutos de retraso y, rara vez, un mensaje repetido. |
| Base local (ADR-001) | Disponibilidad (D1) y simplicidad | Acceso remoto y escalabilidad | El negocio tiene una sede y un mostrador. |
| Un solo perfil sin PIN | Usabilidad para personal con poca experiencia | Seguridad (control de acceso) | Todo el personal hace todo; la auditoría deja rastro de cada cambio. |

### Si el negocio creciera diez veces

Diez veces más turnos en el mismo local no rompen nada: SQLite maneja millones de filas y la agenda se consulta por fecha (N1). Diez veces significa en realidad varias sedes o varios computadores. Lo primero que se rompe es ADR-001, el monolito en un solo equipo (R3), y con él el escritor único de ADR-002. Sobreviven el dominio y los servicios: la frontera de capas permite poner un servidor detrás de la misma capa de servicios sin reescribir las reglas.

## 10 Esqueleto funcional, trazabilidad y funciones de aptitud

El esqueleto es el mismo programa, en `backend/`. Un comando lo recorre de punta a punta sin abrir ventanas y muestra cada decisión en acción:

```
python -m clinican.demo
```

1. **Capas.** Lee el código fuente y muestra de qué capa depende cada una, y que no hay atajos (ADR-001).
2. **D2.** Con una base temporal, dos hilos piden a la vez el último cupo de máquina de las 9:00: uno lo obtiene y el otro recibe el rechazo (ADR-002).
3. **D1.** Se agenda sin red mientras WhatsApp falla; el recordatorio queda con su reintento programado y su error (ADR-003).
4. **D3.** El recordatorio sale por archivo y luego por consola cambiando solo la configuración; la agenda no cambia (ADR-003).

El programa completo, con ventanas, abre con `python main.py`. Las instrucciones para levantarlo están en `INSTRUCCIONES.md` (raíz del repositorio) y el detalle en `backend/README.md`.

### Trazabilidad

| Driver | Decisión | Tácticas | Dónde está en el código | Cómo se comprueba |
|---|---|---|---|---|
| D1 | ADR-001, ADR-003 | T1, T4, T6, T7 | `datos/conexion.py`, `servicios/despachador.py`, `servicios/respaldos.py`, `datos/repo_notificaciones.py` | `test_sin_red.py`, `test_bandeja_y_despachador.py`, `test_respaldos.py`; parte 3 de la demo |
| D2 | ADR-002 | T2, T3, T9 | `dominio/cupos.py`, `dominio/estados.py`, `dominio/abonos.py`, `servicios/agenda.py` | `test_carreras.py`, `test_dominio_cupos.py`, `test_agenda_servicio.py`; parte 2 de la demo |
| D3 | ADR-003 | T5, T6, T8 | Puerto en `servicios/notificaciones.py`; `adaptadores/notificadores.py` y `notificador_*.py`; raíz de composición en `main.py` | `test_adaptadores.py`, `test_arquitectura.py`; partes 1 y 4 de la demo |

### Funciones de aptitud

Pocas reglas, las que protegen decisiones caras de revertir. Corren en cada cambio en GitHub Actions (`.github/workflows/pruebas.yml`), en Windows y Ubuntu, junto con el resto de las pruebas.

| Regla | Verificación automática | Protege |
|---|---|---|
| Las capas solo dependen hacia el dominio | `test_arquitectura.py` analiza los imports con `ast`, incluidos los dinámicos; planta 50 atajos a propósito para demostrar que los detecta | ADR-001 |
| SQL solo en `datos/`; red solo en `adaptadores/` | La misma prueba busca `sqlite3`, `.execute()`, textos SQL y módulos de red fuera de su capa | ADR-001, ADR-003 |
| Ninguna franja supera su cupo | `test_carreras.py` con varios hilos y conexiones | ADR-002 |

| | | |
|---|---|---|
| La agenda del día responde en < 1 s con cinco años de datos | `test_rendimiento.py` (marca `lento`) | N1 |

Lo que no se automatiza (si una responsabilidad está en el módulo correcto, si un cambio merece un ADR) se revisa en el mismo cambio que toca el código: un ADR nuevo va en el mismo commit que lo implementa.

## 11 Cambios respecto a la propuesta y a la versión 1.0

La propuesta se escribió antes de levantar los requisitos con el negocio, y la versión 1.0 de este documento describía un primer prototipo que se reemplazó por el backend actual. Los tres drivers no cambiaron; cambió el modelo donde la realidad era distinta.

| Antes | Quedó así | Por qué |
|---|---|---|
| Agenda por peluquero, con duración variable por raza y servicio | Franjas de 30 minutos con cupos por categoría | El negocio no asigna peluquera al agendar; la variación se expresa en cuántas mascotas caben por franja (ADR-002). |
| Varios tipos de usuario con PIN y permisos (administradora y personal) | Un solo perfil, sin roles ni PIN; el dueño y Meta como actores externos | Recepción, peluquería y administración las hace la misma gente. Se sacrificó control de acceso a cambio de usabilidad; la auditoría queda (sección 9, puntos de compromiso). |
| Módulo legal con términos versionados y autorización de datos (Ley 1581) dentro del sistema | La norma sigue como restricción; la autorización se gestiona fuera del sistema | El negocio no la pidió como flujo del programa. Queda como riesgo R5 por verificar. |
| La categoría dependía de raza, tamaño, pelaje y servicio | Prioridad COMPLICADO > GRANDE > TIJERA > MÁQUINA según la marca de la mascota, su tamaño y el corte | Regla confirmada con el negocio: más simple de explicar y de configurar. |
| Si varios pendientes competían por el último cupo, ganaba el primero que pagaba | El pendiente ocupa su cupo desde que se crea, durante 30 minutos | Regla confirmada con el negocio: quien agenda primero aparta el cupo mientras paga. |